



Actors

Concepts

Actors

An `Actor` receives data, handles events, and manages state. The `Strategy` class extends Actor with order management capabilities.

Key capabilities:

- Data subscription and requests (market data, custom data).
- Event handling and publishing.
- Timers and alerts.
- Cache and portfolio access.
- Logging.

Basic example

Actors support configuration through a pattern similar to strategies.

```
from nautilus_trader.config import ActorConfig
from nautilus_trader.model import InstrumentId
from nautilus_trader.model import Bar, BarType
from nautilus_trader.common.actor import Actor

class MyActorConfig(ActorConfig):
    instrument_id: InstrumentId # example value: "ETHUSD-T-PERP.BINANCE"
    bar_type: BarType          # example value: "ETHUSD-T-PERP.BINANCE-15-MINUTE[LAST]-INTERNAL"
    lookback_period: int = 10

class MyActor(Actor):
    def __init__(self, config: MyActorConfig) -> None:
        super().__init__(config)
```

```
# Custom state variables
self.count_of_processed_bars: int = 0

def on_start(self) -> None:
    # Subscribe to bars matching the configured bar type
    self.subscribe_bars(self.config.bar_type)

def on_bar(self, bar: Bar) -> None:
    self.count_of_processed_bars += 1
```

Actor configuration and IDs

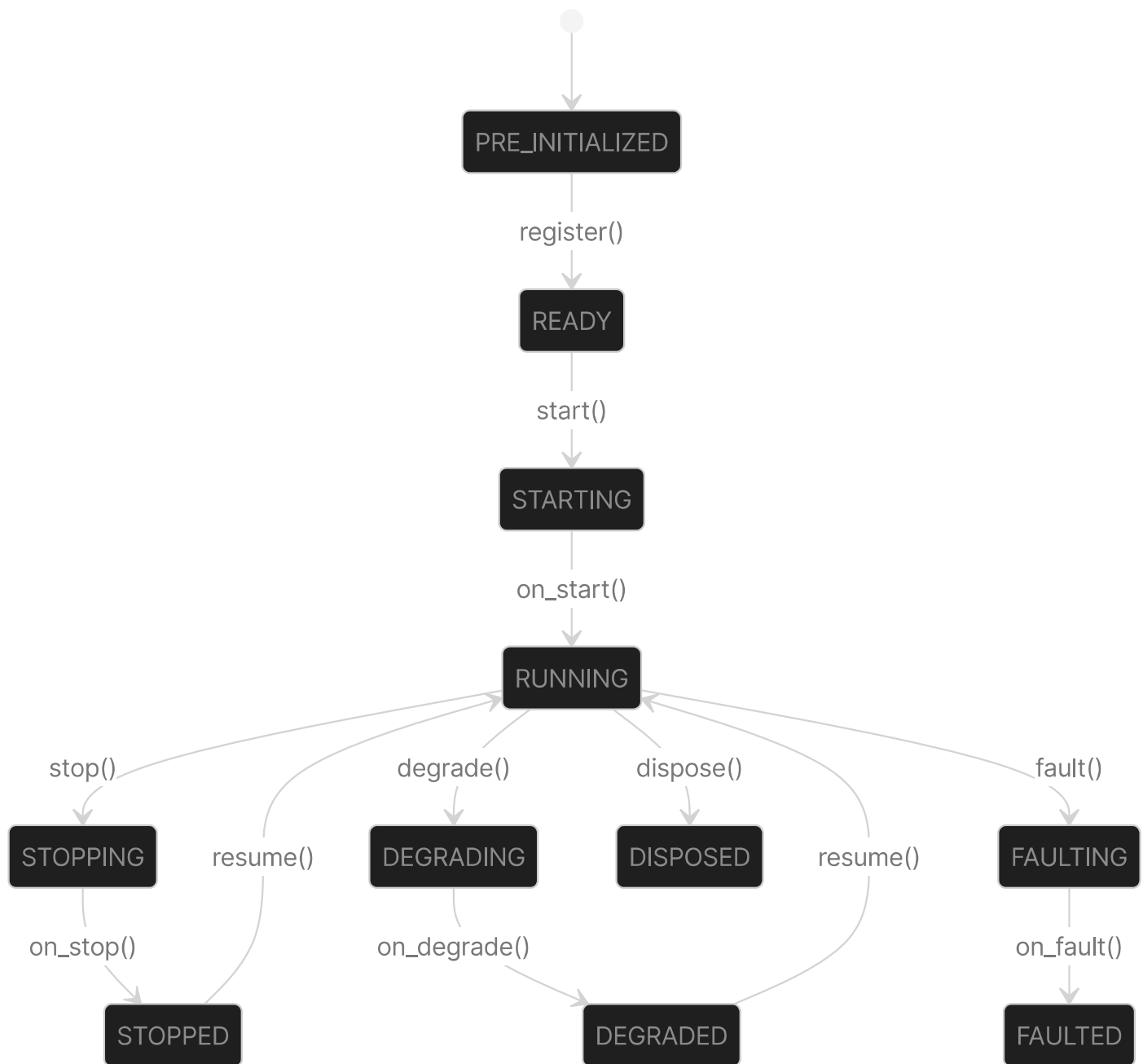
Actors can receive an `ActorConfig` subclass. The base config may include an `actor_id`; if supplied, the actor registers with that ID. If omitted, the system derives a runtime actor ID.

Treat configuration as construction data for the actor. Read user-supplied settings through `self.config`, and keep runtime state on the actor itself.

:::info Rust implementation For Rust actors, generated or assigned runtime IDs live on the actor core rather than being written back into `DataActorConfig`. This differs from Python bridge paths which may copy inherited config fields into runtime state when a Python object is created from an importable config. :::

Lifecycle

Actors follow a defined state machine through their lifecycle:



Override these methods to hook into lifecycle events:

Method	When called
<code>on_start()</code>	Actor is starting (subscribe to data here).
<code>on_stop()</code>	Actor is stopping (cancel timers, clean up resources).
<code>on_resume()</code>	Actor is resuming from a stopped state.
<code>on_reset()</code>	Reset indicators and internal state (called between backtest runs).

Method	When called
<code>on_degrade()</code>	Actor is entering a degraded state (partial functionality).
<code>on_fault()</code>	Actor has encountered a fault.
<code>on_dispose()</code>	Actor is being disposed (final cleanup).

Timers and alerts

Actors have access to a clock for scheduling:

```
def on_start(self) -> None:
    # Set a recurring timer with a callback (fires every 5 seconds)
    self.clock.set_timer(
        "my_timer",
        timedelta(seconds=5),
        callback=self._on_timer,
    )

    # Set a one-time alert with a callback
    self.clock.set_time_alert(
        "my_alert",
        self.clock.utc_now() + timedelta(minutes=1),
        callback=self._on_alert,
    )

def on_stop(self) -> None:
    # Cancel timers to prevent resource leaks across stop/resume cycles
    self.clock.cancel_timer("my_timer")

def _on_timer(self, event: TimeEvent) -> None:
    self.log.info("Timer fired!")

def _on_alert(self, event: TimeEvent) -> None:
    self.log.info("Alert triggered!")
```

Pass a `callback` to direct `TimeEvent` objects to your own method. If you omit the callback, the event is delivered to `on_event` instead.

System access

Actors have access to core system components:

Property	Description
<code>self.cache</code>	Shared state for instruments, orders, positions, etc.
<code>self.portfolio</code>	Portfolio state and calculations.
<code>self.clock</code>	Current time and timer/alert scheduling.
<code>self.log</code>	Structured logging.
<code>self.msgbus</code>	Publish/subscribe to custom messages.

For custom messaging between components, see the [Message Bus](#) guide.

Data handling and callbacks

The system uses different callback handlers depending on whether data is historical or real-time. Understanding the relationship between data *requests/subscriptions* and their handlers is key.

Historical vs real-time data

The system distinguishes between two data flows:

1. Historical data (from *requests*):

- Obtained through methods like `request_bars()`, `request_quote_ticks()`, etc.
- Processed through the `on_historical_data()` handler.
- Used for initial data loading and historical analysis.

2. Real-time data (from *subscriptions*):

- Obtained through methods like `subscribe_bars()`, `subscribe_quote_ticks()`, etc.
- Processed through specific handlers like `on_bar()`, `on_quote_tick()`, etc.

- Used for live data processing.

Callback handlers

Different data operations map to these handlers:

Operation	Category	Handler	Purpose
<code>subscribe_data()</code>	Real-time	<code>on_data()</code>	Live data updates.
<code>subscribe_instrument()</code>	Real-time	<code>on_instrument()</code>	Live instrument definition updates.
<code>subscribe_instruments()</code>	Real-time	<code>on_instrument()</code>	Live instrument definition updates (for venue).
<code>subscribe_order_book_deltas()</code>	Real-time	<code>on_order_book_deltas()</code>	Live order book deltas.
<code>subscribe_order_book_depth()</code>	Real-time	<code>on_order_book_depth()</code>	Live order book depth snapshots.
<code>subscribe_order_book_at_interval()</code>	Real-time	<code>on_order_book()</code>	Live order book snapshots at intervals.
<code>subscribe_quote_ticks()</code>	Real-time	<code>on_quote_tick()</code>	Live quote updates.
<code>subscribe_trade_ticks()</code>	Real-time	<code>on_trade_tick()</code>	Live trade updates.
<code>subscribe_mark_prices()</code>	Real-time	<code>on_mark_price()</code>	Live mark price updates.
<code>subscribe_index_prices()</code>	Real-time	<code>on_index_price()</code>	Live index price updates.
<code>subscribe_bars()</code>	Real-time	<code>on_bar()</code>	Live bar updates.
<code>subscribe_funding_rates()</code>	Real-time	<code>on_funding_rate()</code>	Live funding rate updates.
<code>subscribe_instrument_status()</code>	Real-time	<code>on_instrument_status()</code>	Live instrument status updates.

Operation	Category	Handler	Purpose
<code>subscribe_instrument_close()</code>	Real-time	<code>on_instrument_close()</code>	Live instrument close updates.
<code>subscribe_option_greeks()</code>	Real-time	<code>on_option_greeks()</code>	Live option greeks updates.
<code>subscribe_option_chain()</code>	Real-time	<code>on_option_chain()</code>	Live option chain slice snapshots.
<code>subscribe_order_fills()</code>	Real-time	<code>on_order_filled()</code>	Live order fill events for an instrument.
<code>subscribe_order_cancels()</code>	Real-time	<code>on_order_canceled()</code>	Live order cancel events for an instrument.
<code>request_data()</code>	Historical	<code>on_historical_data()</code>	Historical data processing.
<code>request_order_book_deltas()</code>	Historical	<code>on_historical_data()</code>	Historical order book deltas.
<code>request_order_book_depth()</code>	Historical	<code>on_historical_data()</code>	Historical order book depth.
<code>request_order_book_snapshot()</code>	Historical	<code>on_historical_data()</code>	Historical order book snapshot.
<code>request_instrument()</code>	Historical	<code>on_instrument()</code>	Instrument definition.
<code>request_instruments()</code>	Historical	<code>on_instrument()</code>	Instrument definitions.
<code>request_quote_ticks()</code>	Historical	<code>on_historical_data()</code>	Historical quotes processing.
<code>request_trade_ticks()</code>	Historical	<code>on_historical_data()</code>	Historical trades processing.
<code>request_bars()</code>	Historical	<code>on_historical_data()</code>	Historical bars processing.

Operation	Category	Handler	Purpose
<code>request_aggregated_bars()</code>	Historical	<code>on_historical_data()</code>	Historical aggregated bars (on-the-fly).
<code>request_funding_rates()</code>	Historical	<code>on_historical_data()</code>	Historical funding rates processing.

Example

This example shows both historical and real-time data handling:

```

from nautilus_trader.common.actor import Actor
from nautilus_trader.config import ActorConfig
from nautilus_trader.core.data import Data
from nautilus_trader.model import Bar, BarType
from nautilus_trader.model import ClientId, InstrumentId

class MyActorConfig(ActorConfig):
    instrument_id: InstrumentId # example value: "AAPL.XNAS"
    bar_type: BarType          # example value: "AAPL.XNAS-1-MINUTE-LAST-EXTERNAL"

class MyActor(Actor):
    def __init__(self, config: MyActorConfig) -> None:
        super().__init__(config)
        self.bar_type = config.bar_type

    def on_start(self) -> None:
        # Request historical data - will be processed by on_historical_data() handler
        self.request_bars(
            bar_type=self.bar_type,
            # Many optional parameters
            start=None,          # pd.Timestamp | None
            end=None,           # pd.Timestamp | None
            callback=None,       # Callable[[UUID4], None] | None
            update_catalog_mode=None, # UpdateCatalogMode | None
            params=None,         # dict[str, Any] | None
        )

        # Subscribe to real-time data - will be processed by on_bar() handler
        self.subscribe_bars(
            bar_type=self.bar_type,

```


Separating historical and real-time handlers lets you apply different processing logic based on context. For example:

- Use historical data to initialize indicators or establish baseline metrics.
- Process real-time data differently for live trading decisions.
- Apply different validation or logging for historical vs real-time data.



When debugging data flow issues, check that you're looking at the correct handler for your data source. If you're not seeing data in `on_bar()` but see log messages about receiving bars, check `on_historical_data()` as the data might be coming from a request rather than a subscription.

Order fill subscriptions

Actors can subscribe to order fill events for specific instruments using `subscribe_order_fills()`. This is useful for monitoring trading activity, fill analysis, or tracking execution quality.

When subscribed, the handler `on_order_filled()` receives all fills for the specified instrument, regardless of which strategy or component generated the original order.

Example

```
from nautilus_trader.common.actor import Actor
from nautilus_trader.config import ActorConfig
from nautilus_trader.model import InstrumentId
from nautilus_trader.model.events import OrderFilled

class MyActorConfig(ActorConfig):
    instrument_id: InstrumentId # example value: "ETHUSD-T-PERP.BINANCE"

class FillMonitorActor(Actor):
    def __init__(self, config: MyActorConfig) -> None:
        super().__init__(config)
        self.fill_count = 0
        self.total_volume = 0.0

    def on_start(self) -> None:
        # Subscribe to all fills for the instrument
        self.subscribe_order_fills(self.config.instrument_id)
```

```
def on_order_filled(self, event: OrderFilled) -> None:
    # Handle order fill events
    self.fill_count += 1
    self.total_volume += float(event.last_qty)

    self.log.info(
        f"Fill received: {event.order_side} {event.last_qty} @ {event.last_px}, "
        f"Total fills: {self.fill_count}, Volume: {self.total_volume}"
    )

def on_stop(self) -> None:
    # Unsubscribe from fills
```



Order fill subscriptions use the message bus only and do not involve the data engine. The `on_order_filled()` handler receives events only while the actor is running.

Order cancel subscriptions

Actors can subscribe to order cancel events for specific instruments using `subscribe_order_cancels()`. This is useful for monitoring cancellations or tracking order lifecycle events.

When subscribed, the handler `on_order_canceled()` receives all cancels for the specified instrument, regardless of which strategy or component generated the original order.

Example

```
from nautilus_trader.common.actor import Actor
from nautilus_trader.config import ActorConfig
from nautilus_trader.model import InstrumentId
from nautilus_trader.model.events import OrderCanceled

class MyActorConfig(ActorConfig):
    instrument_id: InstrumentId # example value: "ETHUSDT-PERP.BINANCE"

class CancelMonitorActor(Actor):
    def __init__(self, config: MyActorConfig) -> None:
        super().__init__(config)
        self.cancel_count = 0

    def on_start(self) -> None:
```



```
# Subscribe to all cancels for the instrument
self.subscribe_order_cancels(self.config.instrument_id)

def on_order_canceled(self, event: OrderCanceled) -> None:
    # Handle order cancel events
    self.cancel_count += 1

    self.log.info(
        f"Cancel received: {event.client_order_id}, "
        f"Total cancels: {self.cancel_count}"
    )

def on_stop(self) -> None:
    # Unsubscribe from cancels
    self.unsubscribe_order_cancels(self.config.instrument_id)
```



Order cancel subscriptions use the message bus only and do not involve the data engine. The `on_order_canceled()` handler receives events only while the actor is running.

Related guides

- [Strategies](#) - Strategies extend actors with order management capabilities.
- [Data](#) - Data types and subscriptions available to actors.
- [Message Bus](#) - The messaging system actors use for communication.

[< Architecture](#)[Previous Page](#)[Strategies >](#)[Next Page](#)